

Student

Chapter 1 gave me the foundation. Now I want to actually **design analytical models like a professional**. Where do I start?

ChatGPT • AI Assistant

With **Ralph Kimball's dimensional modeling** - the methodology that powers most modern warehouses. This chapter teaches:

1. What **Kimball dimensional modeling** actually is.
2. **Fact tables** - where the measures live.
3. **Dimension tables** - where the descriptive context lives.
4. **Grain** - the single most important design decision you'll make.
5. Three types of **fact tables** - transaction, periodic snapshot, accumulating snapshot.
6. **Conformed dimensions** and the **Kimball bus architecture**.
7. Special dimensions - **degenerate, junk, and role-playing**.
8. A **hands-on transformation** of Chapter 1's OLTP model into a star schema.

CONCEPT CARD

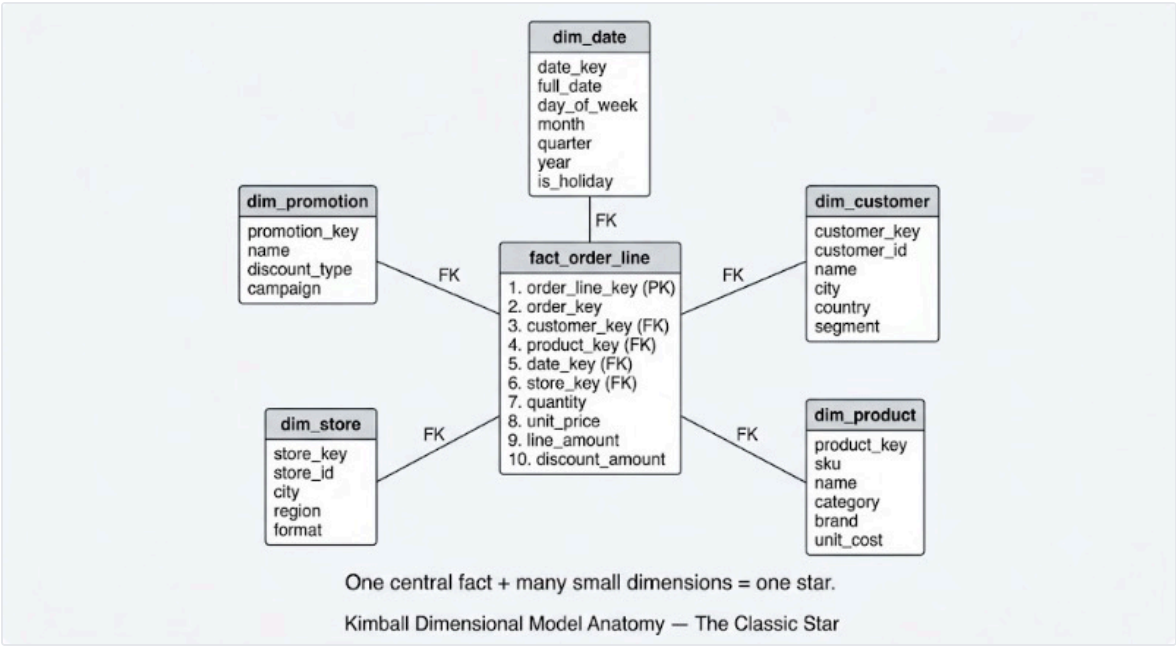
Kimball dimensional modeling is a design approach that organizes analytical data into two kinds of tables: **facts** (measurable events, mostly numbers) and **dimensions** (descriptive context, mostly text). It's **bottom-up** (start with a single business process), **business-process driven**, and famously easier to teach and query than 3NF warehouses.

SUMMARY CARD

By the end of this chapter you'll be able to:

- Design a star schema for any business process in four steps.
- Declare grain correctly on the first try.
- Pick the right fact-table type for your scenario.
- Reuse conformed dimensions across multiple facts.
- Convert a normalized OLTP model into a warehouse-ready star.

[AI Prompt] Warm-up (paste into any AI chatbot): *"Explain Ralph Kimball's four-step dimensional design process. For each step (business process → grain → dimensions → facts), give a concrete example from an e-commerce checkout flow."*



Kimball Dimensional Model Anatomy — The Classic Star

Facts vs Dimensions — The Core Distinction		
Aspect	Fact Table	Dimension Table
Represents	A measurable event or activity	Descriptive context (who/what/where/when)
Content	Mostly numeric measures + FKs	Mostly text attributes
Size	Millions to billions of rows	Thousands to millions
Width	Narrow — few columns	Wide — many columns
Grain	One row per event (defined explicitly)	One row per entity
Keys	Composite of FKs to dimensions	One surrogate PK
Change frequency	Insert-only, high volume	Slowly changing

Aspect	Fact Table	Dimension Table
Naming	fact_<business_process>	dim_<subject>

sql – A minimal star schema – one fact + four dimensions

```
-- =====
-- Star schema for retail sales – the reference example
-- =====

CREATE TABLE dim_date (
    date_key      INT          PRIMARY KEY,      -- YYYYMMDD
    full_date     DATE          NOT NULL,
    day_of_week   VARCHAR(10),
    month_name    VARCHAR(10),
    quarter       INT,
    year          INT,
    is_holiday    BOOLEAN      DEFAULT FALSE
);

CREATE TABLE dim_customer (
    customer_key  BIGINT       PRIMARY KEY,
    customer_id   VARCHAR(50)  NOT NULL,
    full_name     VARCHAR(200),
    city          VARCHAR(100),
    country       VARCHAR(80),
    segment       VARCHAR(50)   -- new/loyal/vip
);

CREATE TABLE dim_product (
    product_key   BIGINT       PRIMARY KEY,
    sku           VARCHAR(50)  NOT NULL,
    product_name  VARCHAR(200),
    category     VARCHAR(100),
    brand        VARCHAR(100),
    unit_cost     DECIMAL(12, 2)
);

CREATE TABLE dim_store (
    store_key     BIGINT       PRIMARY KEY,
    store_id      VARCHAR(20)  NOT NULL,
    store_city    VARCHAR(100),
    region        VARCHAR(50),
    store_format  VARCHAR(20)   --
    flagship/express/online
```

```

);

CREATE TABLE fact_sales (
    sales_key      BIGINT      PRIMARY KEY,
    date_key       INT         REFERENCES dim_date(date_key),
    customer_key   BIGINT      REFERENCES dim_customer(customer_key),
    product_key    BIGINT      REFERENCES dim_product(product_key),
    store_key      BIGINT      REFERENCES dim_store(store_key),
    order_id       VARCHAR(50), -- degenerate dimension
    quantity       INT         NOT NULL,
    unit_price     DECIMAL(12, 2) NOT NULL,
    line_amount    DECIMAL(12, 2) NOT NULL,
    discount_amt   DECIMAL(12, 2) DEFAULT 0
);

-- Indexes on FKs speed up star joins dramatically
CREATE INDEX idx_fs_date      ON fact_sales(date_key);
CREATE INDEX idx_fs_customer ON fact_sales(customer_key);
CREATE INDEX idx_fs_product  ON fact_sales(product_key);
CREATE INDEX idx_fs_store    ON fact_sales(store_key);

```

WHAT IS THE KIMBALL METHODOLOGY?

Student

Who is **Ralph Kimball**, and why does everyone use his methodology?

ChatGPT • AI Assistant

Ralph Kimball is a data-warehouse pioneer whose 1996 book *The Data Warehouse Toolkit* introduced **dimensional modeling** as a practical alternative to Inmon's enterprise-wide 3NF warehouses. The Kimball approach is **bottom-up, business-process focused**, and famously easier to teach, build, and query.

CONCEPT CARD

The four Kimball principles:

1. **Business process first** - model one business process at a time (Sales, Inventory, Support, etc.).
2. **Grain is sacred** - declare the level of detail before touching a column.
3. **Dimensions and facts are separate** - descriptive context lives in dimensions; measurable events live in facts.
4. **Conformed dimensions** - the same customer / product / date table is reused across many facts.

Student

What's the **4-step design process**?

ChatGPT • AI Assistant

EXAMPLE CARD

Kimball's classic four-step design (applied to "we want to analyze retail sales"):

1. **Select the business process** → Retail sales checkouts.
2. **Declare the grain** → One row per line item on a sales receipt.
3. **Identify the dimensions** → Date, customer, product, store, promotion.
4. **Identify the facts** → Quantity, unit price, line amount, discount amount.

Write these four decisions down before touching any DDL. That single discipline avoids 90% of the design mistakes beginners make.

🔗 ANALOGY

Kimball is like **cooking a meal by course** - you plan the appetizer first, then the main, then dessert, each done well. Inmon is like **building an entire commercial kitchen** before serving a single dish - powerful but overkill for most teams.

⚠️ WARNING

Kimball isn't a religion. Modern lakehouses often use one big table (OBT) or hybrid layouts. But even in those, the Kimball vocabulary (fact, dimension, grain, conformed) remains the shared language of analytics teams.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Jumping to physical tables before deciding grain. You build the star, then discover you can't answer half your KPIs.

Correct Way: Do the 4-step design on a whiteboard first. Write down grain in one sentence. Then build.

EXAM TIP CARD

Interview classic: *"Walk me through Kimball's four-step dimensional design."*

Winning answer: *"Business process → grain → dimensions → facts. Each step depends on the previous, and grain is the pivotal decision."*

EXERCISE

Pick any domain you know (a library, a gym, a school). Do the 4-step Kimball design in 15 minutes: pick a process, declare grain, list dimensions, list facts. That's a complete first-draft dimensional model.

[AI Prompt] Paste this into any AI chatbot: *"Apply Kimball's four-step design to a hospital admissions system. Show me the business process, grain, dimensions, and facts - with a brief justification for each choice."*

? PRACTICE QUIZ

Which Kimball design step is considered the pivotal one?

Answer: Declaring the grain - everything else (dimensions, facts, queries) depends on it.

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

FACT TABLES — MEASURES AND FOREIGN KEYS

ChatGPT • AI Assistant

sql — A well-designed fact table — mostly FKs and additive measures

```
CREATE TABLE fact_sales (  
  -- Surrogate PK (optional but recommended for merges)  
  sales_key      BIGINT      PRIMARY KEY,  
  
  -- Foreign keys to dimensions (the "points" of the star)  
  date_key       INT         NOT NULL  
                  REFERENCES dim_date(date_key),  
  customer_key   BIGINT      NOT NULL  
                  REFERENCES dim_customer(customer_key),
```

```

product_key    BIGINT      NOT NULL
                REFERENCES dim_product(product_key),
store_key      BIGINT      NOT NULL
                REFERENCES dim_store(store_key),
promotion_key  BIGINT
                REFERENCES dim_promotion(promotion_key),

-- Degenerate dimension (an ID with no descriptive dim)
order_id       VARCHAR(50) NOT NULL,

-- Additive measures
quantity       INT         NOT NULL,
gross_amount   DECIMAL(12, 2) NOT NULL,
discount_amount DECIMAL(12, 2) NOT NULL DEFAULT 0,
net_amount     DECIMAL(12, 2) NOT NULL,
tax_amount     DECIMAL(12, 2) NOT NULL DEFAULT 0,

-- Non-additive: keep the atomic value only
unit_price     DECIMAL(12, 2) NOT NULL
);

```

Student

What exactly goes in a **fact table**?

ChatGPT • AI Assistant

Two kinds of columns: **measures** (numeric facts you can sum/average) and **foreign keys** to dimensions. That's almost it. Fact tables are **narrow but tall** - few columns, millions of rows.

CONCEPT CARD

What lives in a fact table:

1. **Foreign keys to dimensions** - one per contextual axis (customer, product, date, store...).
2. **Additive measures** - sums make sense (quantity, revenue, cost).
3. **Semi-additive measures** - sums work across some dimensions but not all (inventory can sum across products, not across days).
4. **Non-additive measures** - sums make no sense (ratios, percentages, unit_price).

5. **Degenerate dimensions** - attributes with no dimension table (*orderid*, *transactionid*).

What does NOT belong in a fact table:

1. Long text descriptions (put them in a dimension).
2. Business-key strings if the surrogate FK is available.
3. Attributes that describe an entity - those go in dimensions.

Student

How do I tell if a measure is additive?

ChatGPT • AI Assistant

EXAMPLE CARD

The classic tests:

1. **Additive** - Can I sum this across ANY dimension? (Sales revenue: sum across days, products, stores - all fine.)
2. **Semi-additive** - Sum works across SOME dimensions but not time. (Account balance: sum across accounts on ONE day - makes sense; sum across days - nonsense.)
3. **Non-additive** - Never sum this. Ratios like *conversion_rate* or *unit_price* must be recomputed from base measures.

🔗 ANALOGY

A fact table is like a **cash-register receipt log** - one row per line item, with numeric amounts and pointers back to who bought what, where, and when. Descriptive text ("large blue T-shirt, cotton") belongs in the product catalog, not in the receipt log.

⚠️ WARNING

Never store **calculated aggregates** in the base fact table. Store the **atomic measures** and let BI tools compute totals, averages, and ratios on demand. Pre-aggregated tables are separate and named `agg_...`.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Putting `customer_name` and `product_description` directly in the fact table "to avoid joins." The fact bloats; when the name changes, history breaks.

Correct Way: Fact keeps only FKs + measures. Descriptive text lives in the dimension where it can be updated via SCD (Chapter 3).

EXAM TIP CARD

Interview classic: *"What are the three types of measures in a fact table?"* Winning answer: *"Additive (sum anywhere), semi-additive (sum in some dimensions, not others), and non-additive (never sum - like ratios or unit prices)."*

🔥 EXERCISE

Look at your Chapter 1 e-commerce model. For each numeric column, classify it: additive, semi-additive, or non-additive. Move any descriptive text out into dimensions.

[AI Prompt] Paste this into any AI chatbot: *"For a fact_sales table with columns quantity, unit_price, line_amount, discount_percent, running_total, and account_balance_at_time, classify each measure as additive, semi-additive, or non-additive. Explain each."*

DIMENSION TABLES — DESCRIPTIVE CONTEXT



ChatGPT • AI Assistant

```
sql - A wide, denormalized dimension table - the professional pattern

CREATE TABLE dim_product (
  -- Keys
  product_key          BIGINT          PRIMARY KEY,
  sku                  VARCHAR(50)    NOT NULL,          -- natural key

  -- Identity
  product_name         VARCHAR(200) NOT NULL,
  short_description    VARCHAR(500),

  -- Product hierarchy (flat, denormalized)
  category             VARCHAR(100),
  sub_category         VARCHAR(100),
  department           VARCHAR(100),
  division             VARCHAR(100),

  -- Brand and supply
  brand               VARCHAR(100),
  manufacturer         VARCHAR(200),
  supplier_code       VARCHAR(50),
```

```

-- Physical attributes
color          VARCHAR(30),
size           VARCHAR(20),
weight_kg      DECIMAL(8, 3),
package_type   VARCHAR(30),

-- Pricing snapshot
list_price     DECIMAL(12, 2),
unit_cost      DECIMAL(12, 2),
standard_margin_pct DECIMAL(5, 2),

-- Behavioral flags (updated by ETL)
is_active      BOOLEAN      DEFAULT TRUE,
is_seasonal    BOOLEAN      DEFAULT FALSE,
is_private_label BOOLEAN      DEFAULT FALSE,

-- Lifecycle dates
first_sold_date DATE,
discontinued_date DATE,

-- SCD Type 2 columns (Chapter 3)
effective_from DATE          NOT NULL,
effective_to   DATE,
is_current     BOOLEAN      NOT NULL DEFAULT TRUE,

-- Audit
updated_ts     TIMESTAMP     NOT NULL DEFAULT
CURRENT_TIMESTAMP,
source_system  VARCHAR(50)
);

```

Student

What goes in a **dimension table**?

ChatGPT • AI Assistant

A dimension is **all the descriptive context** for one subject - customer, product, store, date. Dimensions answer the **who/what/where/when/why/how** questions that give facts their meaning.

They're **wide** (many columns) and **relatively small** (thousands to millions of rows).

CONCEPT CARD

Anatomy of a good dimension table:

1. **Surrogate PK** (`customer_key BIGINT`).
2. **Natural key** as attribute (`customer_id VARCHAR(50) UNIQUE`).
3. **Descriptive attributes** - as many as you need. Denormalized is fine.
4. **Hierarchy attributes** - city → state → country in the same row.
5. **Behavioral attributes** - segment, tier, loyalty status (updated by ETL).
6. **SCD columns** (Chapter 3) - *effectivefrom*, *effectiveto*, *is_current*.
7. **Audit columns** - *updatedts*, *sourcesystem*.

Student

How wide should a dimension be?

ChatGPT • AI Assistant

EXAMPLE CARD

Wider is better in dimensions:

1. **50-200 columns is common** for major dimensions like `dim_customer` and `dim_product` .
2. Every attribute that describes the entity, EVEN IF derived, belongs here.
3. Storing hierarchies flat (`city`, `state`, `country` in the same row) makes drill-down queries trivial.
4. Text attributes should be human-readable - no cryptic codes without the label alongside.

🔗 ANALOGY

A dimension is like a **rich Wikipedia article** about an entity. Include everything a reader might want to know: description, category, region, status, dates, and more. The fact table is just a **timestamped bookmark** to the article.

⚠️ WARNING

Never store the **same attribute in different tables**. `product_category` lives in `dim_product` - not in `fact_sales` , not in a separate `dim_category` (that would be a snowflake - Chapter 4).

🚫 COMMON PITFALL / MISTAKE

Incorrect Way: A *dimproduct* with 5 columns and a *dimcategory* with 10 columns joined via FK. Every product query requires an extra join.

Correct Way: Denormalize category attributes INTO *dim_product*. Bigger dim, faster queries.

EXAMPLE CARD

Common dimensions almost every warehouse needs:

1. **dim_date** - one row per calendar day (Section 2-7).
2. **dim_customer** - user attributes, segments, lifetime metrics.
3. **dim_product** - SKU, category, brand, cost.
4. **dim_store** - physical or virtual location.
5. **dim_employee** - for HR or sales-attribution.
6. **dim_geography** - city / state / country / region hierarchy.

EXAM TIP CARD

Interview classic: *"How wide should a dimension table be?"* Winning answer: *"As wide as needed. Denormalizing category, region, brand, and hierarchy attributes into one dimension trades storage for query simplicity - the core Kimball trade-off."*

🔥 EXERCISE

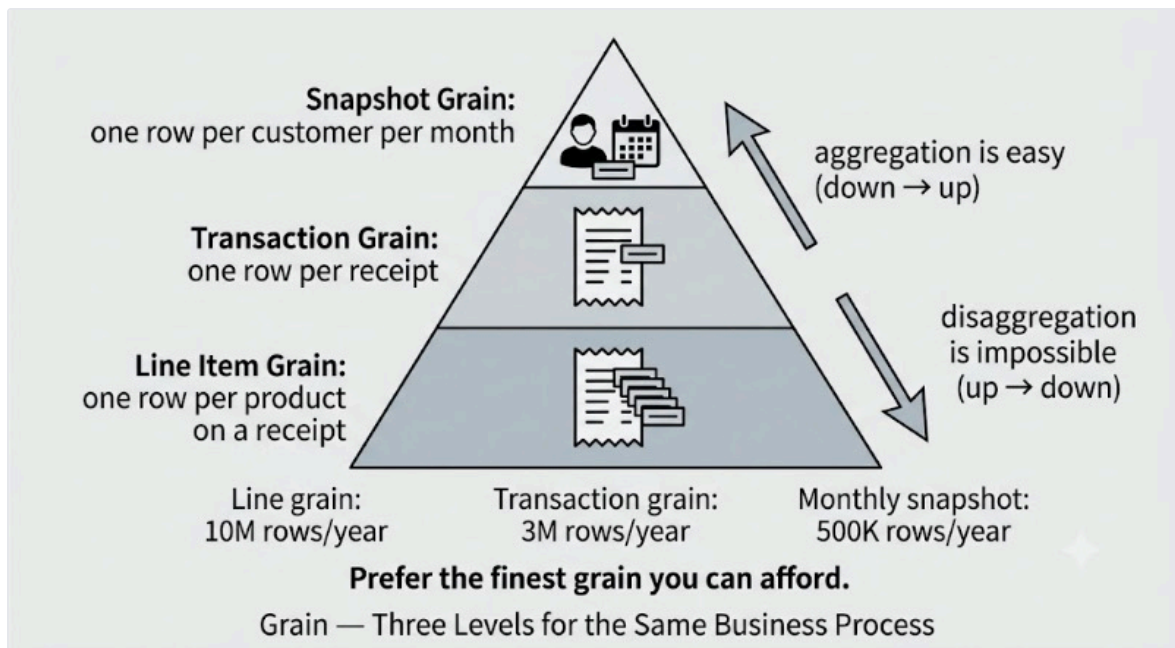
Take your Chapter 1 **customer** table. Widen it: add city, state, country, segment, *lifetimevalue*, *firstorderdate*, *isactive*. That's the beginning of your **dim_customer**.

[AI Prompt] Paste this into any AI chatbot: *"Design a rich dim_product for an electronics retailer. Include 20-30 attributes covering identity, hierarchy, physical properties, pricing history, and behavioral flags. Show the DDL."*

GRAIN — THE SINGLE MOST IMPORTANT DECISION



ChatGPT • AI Assistant



Grain — Three Levels for the Same Business Process

Student

You keep saying **grain** is the most important decision. What is it?

ChatGPT • AI Assistant

Grain is the answer to one question: "**What does one row in the fact table represent?**" It's the finest level of detail captured. Get grain right, and everything else falls into place. Get it wrong, and no query works correctly.

CONCEPT CARD

Grain must be a **single, precise sentence**:

- ✓ "One row per line item on a sales receipt."
- ✓ "One row per customer per day, ending in a balance snapshot."
- ✓ "One row per support ticket, updated at each status change."
- ✗ "Sales data" - not a grain, that's a topic.
- ✗ "Everything about customers" - too vague.

Student

What are the three main grain levels for retail sales?

ChatGPT • AI Assistant

EXAMPLE CARD

For the same retail domain, three legitimate grains:

1. **Line item** - one row per product on a receipt. Finest grain. Enables product-level analysis.
2. **Transaction / receipt** - one row per receipt (aggregate of lines). Loses per-product detail.
3. **Daily / monthly snapshot** - one row per customer per period. Great for periodic reporting.

Prefer the finest grain you can afford. You can always aggregate up; you can never disaggregate down.

🔗 ANALOGY

Grain is like the **resolution of a photo**. A 20-megapixel photo (fine grain) can be shrunk to any size. A 100-pixel thumbnail (coarse grain) can never be enlarged. Always capture at the highest resolution the business needs, then aggregate for reports.

⚠️ WARNING

Mixing grains in one fact table is the deadliest sin in dimensional modeling. If some rows are line items and others are totals, all your sums double-count. Every fact table must have ONE consistent grain.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: `fact_sales` with some rows at line-item grain and some at receipt-total grain. Any `SUM(net_amount)` triples reality.

Correct Way: One fact per grain. If you need both, build two tables - `fact_sales_line` and `fact_sales_daily`.

EXAM TIP CARD

Interview classic: *"Why is declaring grain so critical?"* Winning answer: *"Grain fixes what one row means. Every FK, measure, and query depends on that definition. Mixing grains double-counts; skipping grain declaration leads to unusable reports."*

🔥 EXERCISE

For any fact you'd model in your team's business, write the grain as a **one-sentence declaration**. Read it aloud. If it doesn't uniquely identify what one row is, refine it.

[AI Prompt] Paste this into any AI chatbot: *"For a hotel bookings analytics system, propose three legitimate grains for a fact table (line-item, transaction-level, periodic snapshot). For each, list the questions it can and cannot answer."*

TYPES OF FACT TABLES — TRANSACTION, PERIODIC SNAPSHOT, ACCUMULATING

ChatGPT • AI Assistant

sql – The three fact-table types side by side

```
-- ===== 1) TRANSACTION FACT (insert-only) =====
CREATE TABLE fact_sales (
    sales_key      BIGINT PRIMARY KEY,
    date_key       INT      NOT NULL,
    customer_key   BIGINT NOT NULL,
    product_key    BIGINT NOT NULL,
    store_key      BIGINT NOT NULL,
    quantity       INT      NOT NULL,
    net_amount     DECIMAL(12, 2) NOT NULL
);

-- ===== 2) PERIODIC SNAPSHOT FACT =====
-- One row per SKU per store per day
CREATE TABLE fact_inventory_daily (
    snapshot_date_key INT      NOT NULL,
    product_key       BIGINT NOT NULL,
    store_key         BIGINT NOT NULL,
    units_on_hand     INT      NOT NULL,          -- semi-additive
    units_on_order    INT      NOT NULL,
    inventory_value   DECIMAL(14, 2) NOT NULL,
    PRIMARY KEY (snapshot_date_key, product_key, store_key)
);

-- ===== 3) ACCUMULATING SNAPSHOT FACT =====
-- One row per order – UPDATED as it moves through milestones
CREATE TABLE fact_order_lifecycle (
    order_key      BIGINT PRIMARY KEY,
```

```

customer_key      BIGINT NOT NULL,
order_id          VARCHAR(50) NOT NULL,

-- Milestone date FKs (nullable until they occur)
order_placed_date_key  INT,
payment_captured_date_key INT,
packed_date_key        INT,
shipped_date_key       INT,
delivered_date_key     INT,
returned_date_key      INT,

-- Elapsed-time measures (updated on each milestone)
hours_to_payment      DECIMAL(8, 2),
hours_to_ship         DECIMAL(8, 2),
hours_to_deliver      DECIMAL(8, 2),

-- Final measures
total_amount          DECIMAL(12, 2),
current_status        VARCHAR(20)
);

```

Student

Are all fact tables the same shape?

ChatGPT • AI Assistant

No - there are **three canonical types**, each answering a different kind of business question:

1. **Transaction fact** - one row per event when it happens.
2. **Periodic snapshot fact** - one row per entity per fixed time period (daily, monthly).
3. **Accumulating snapshot fact** - one row per process instance, updated as it progresses through milestones.

CONCEPT CARD

Quick decision guide:

- Selling something / clicking / logging in → **Transaction**.
- Balance at end of day / inventory at end of month → **Periodic snapshot**.

- Order lifecycle from placed → shipped → delivered → returned → **Accumulating snapshot**.

Student

Give me a concrete example of each.

ChatGPT • AI Assistant

EXAMPLE CARD

The three types side by side:

1. **Transaction:** ``fact_sales`` - one row per line item at checkout. Insert-only.
2. **Periodic snapshot:** ``fact_inventory_daily`` - one row per SKU per store per day, capturing units on hand at day's end.
3. **Accumulating snapshot:** ``fact_order_lifecycle`` - one row per order with columns for `order_placed_date_key` , `paid_date_key` , `shipped_date_key` , `delivered_date_key` , `returned_date_key` . Row is UPDATED as the order progresses.

🔗 ANALOGY

Think of the three types as different views of a football game:

1. **Transaction** - every pass, kick, and goal logged as it happens.
2. **Periodic snapshot** - the score at the end of each quarter.
3. **Accumulating snapshot** - the game's timeline: kickoff → first-half → half-time → second-half → full-time, updated as each phase completes.

⚠️ WARNING

Accumulating snapshots are the only fact type where you **update rows in place**. In the other two, once a row is written, it's never modified.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Using a transaction fact to answer inventory questions. To find "how many units did we have at end of Q3?" you have to replay every stock movement - expensive and error-prone.

Correct Way: Model that as a periodic snapshot fact - one row per SKU per day. The answer is a single indexed lookup.

EXAM TIP CARD

Interview classic: *"When would you use an accumulating snapshot fact?"* Winning answer: *"For processes with well-defined milestones - order fulfillment, loan approvals, hospital admissions. Each row is one process instance, updated as it moves through stages."*

🔥 EXERCISE

For any business you know, list one candidate for each fact type: a transaction (what event?), a periodic snapshot (what entity, what period?), and an accumulating snapshot (what process, what milestones?).

[AI Prompt] Paste this into any AI chatbot: *"For an e-commerce business, propose one transaction fact, one periodic snapshot fact, and one accumulating snapshot fact. Describe grain, key columns, and one KPI each unlocks."*

CONFORMED DIMENSIONS — THE BUS ARCHITECTURE

ChatGPT • AI Assistant

	dim_date	dim_customer	dim_product	dim_store	dim_employee	dim_geography
fact_sales	✓	✓	✓	✓	✓	✓
fact_inventory_daily	✓		✓	✓		✓
fact_support_ticket	✓	✓	✓		✓	
fact_web_session	✓	✓				✓
fact_returns	✓	✓	✓	✓		

Same customer_key = same customer everywhere. Cross-fact joins just work.

The Kimball Bus Matrix — Fact Processes × Conformed Dimensions

The Kimball Bus Matrix — Fact Processes × Conformed Dimensions

sql – Cross-fact analysis via conformed dimensions

```
-- Question: For each customer segment, what's the ratio of
-- support tickets to sales orders in the last quarter?
-- Answer needs TWO facts joined on the SAME dim_customer.
```

```

WITH sales_by_segment AS (
    SELECT c.segment,
           COUNT(DISTINCT s.order_id) AS orders
    FROM   fact_sales s
    JOIN   dim_customer c ON c.customer_key = s.customer_key
    JOIN   dim_date      d ON d.date_key      = s.date_key
    WHERE  d.year = 2026 AND d.quarter = 2
    GROUP BY c.segment
),
tickets_by_segment AS (
    SELECT c.segment,
           COUNT(*) AS tickets
    FROM   fact_support_ticket t
    JOIN   dim_customer c ON c.customer_key = t.customer_key
    JOIN   dim_date      d ON d.date_key      = t.opened_date_key
    WHERE  d.year = 2026 AND d.quarter = 2
    GROUP BY c.segment
)
SELECT s.segment,
       s.orders,
       COALESCE(t.tickets, 0) AS tickets,
       ROUND(COALESCE(t.tickets, 0) * 1.0 / s.orders, 3)
         AS tickets_per_order
FROM   sales_by_segment s
LEFT JOIN tickets_by_segment t ON t.segment = s.segment
ORDER BY tickets_per_order DESC;

-- The join works because BOTH facts point to the SAME
-- dim_customer. That is conformity in action.

```

Student

What are **conformed dimensions** and why do they matter?

ChatGPT • AI Assistant

A **conformed dimension** is a dimension table that has **the same structure and content across every fact table that references it**. When `dim_customer` is conformed, the same `customer_key` means the same customer everywhere - in `fact_sales`, `fact_support_ticket`, `fact_web_session`, and any future fact.

CONCEPT CARD

Why conformed dimensions are the secret to warehouse scalability:

1. **Cross-process analysis** - you can join `fact_sales` with `fact_support` on `customer_key` because both use the same `dim_customer`.
2. **Single source of truth** - one place to update customer attributes.
3. **Team scaling** - different teams can build different facts and know they'll integrate.
4. **Trust** - one "active customer count" everyone agrees on.

Student

What's the **Kimball bus**?

ChatGPT • AI Assistant

EXAMPLE CARD

The **Kimball bus** is a **matrix** - one row per business process, one column per conformed dimension. A tick in a cell means "this fact uses this dimension." The bus becomes the map of the whole warehouse.

Example bus:

	Date	Customer	Product	Store	Employee	
	-----	----	-----	-----	-----	-----
Sales	✓	✓	✓	✓	✓	
Inventory	✓		✓	✓		
Support tickets	✓	✓	✓		✓	
Web sessions	✓	✓				

Every fact that ticks the same dimension **MUST** use the exact same dim table.

🔗 ANALOGY

The bus is like a **city's public transport network**. Every route shares the same bus-stop platform (the conformed dimension). Passengers (queries) can transfer between routes at any shared stop.

⚠️ WARNING

Conforming dimensions requires **discipline across teams**. If one team creates a private `customer_dim`, the whole warehouse fragments - and you lose the ability to answer cross-process questions.

⚠️ COMMON PITFALL / MISTAKE

Incorrect Way: Every team builds their own `dim_customer` . Six months later, no one can produce a single trusted customer count.

Correct Way: One central `dim_customer` , owned by a data platform team, versioned with SCD Type 2, referenced by every fact.

EXAM TIP CARD

Interview favorite: *"What is a conformed dimension?"* Winning answer: *"A dimension used identically across multiple fact tables - same structure, same content, same surrogate keys. It's what enables cross-process analysis and single-source-of-truth reporting."*

🔥 EXERCISE

Sketch a Kimball bus for a company you know. List 3-5 business processes as rows and 5-8 dimensions as columns. Put ticks where each fact needs each dimension. That's your warehouse roadmap.

[AI Prompt] Paste this into any AI chatbot: *"Design a Kimball bus matrix for an airline. Rows = business processes (bookings, flights, boardings, complaints, loyalty).*

Columns = conformed dimensions (date, passenger, flight, aircraft, airport, employee). Show the matrix with checkmarks."

DATE AND TIME DIMENSIONS

ChatGPT • AI Assistant

sql - A production-grade dim_date table

```
CREATE TABLE dim_date (  
  date_key          INT          PRIMARY KEY,    -- YYYYMMDD  
  full_date         DATE         NOT NULL UNIQUE,  
  
  -- Day parts  
  day_of_month      INT          NOT NULL,  
  day_of_week       INT          NOT NULL,       -- 1=Mon .. 7=Sun  
  day_name          VARCHAR(10),    -- Monday .. Sunday  
  day_short         VARCHAR(3),     -- Mon .. Sun  
  day_of_year       INT          NOT NULL,
```

```

-- Week parts
week_of_year      INT      NOT NULL,
iso_week          INT      NOT NULL,

-- Month parts
month_number      INT      NOT NULL,      -- 1..12
month_name        VARCHAR(10),           -- January..
month_short       VARCHAR(3),            -- Jan..Dec
year_month        CHAR(7),               -- '2026-07'

-- Quarter
quarter_number    INT      NOT NULL,      -- 1..4
quarter_name      VARCHAR(10),           -- 'Q3 2026'

-- Year
year              INT      NOT NULL,

-- Fiscal (example: fiscal year starts April 1)
fiscal_year       INT,
fiscal_quarter    INT,
fiscal_month      INT,

-- Flags
is_weekend        BOOLEAN NOT NULL DEFAULT FALSE,
is_holiday        BOOLEAN NOT NULL DEFAULT FALSE,
is_business_day   BOOLEAN NOT NULL DEFAULT TRUE,
is_month_end      BOOLEAN NOT NULL DEFAULT FALSE,
is_quarter_end    BOOLEAN NOT NULL DEFAULT FALSE,
is_year_end       BOOLEAN NOT NULL DEFAULT FALSE,

-- Handy prior-period pointers
prior_day_key      INT,
same_day_last_week_key  INT,
same_day_last_month_key INT,
same_day_last_year_key  INT
);

-- Example: reports read calendar via join, never via functions
SELECT d.year_month,
       SUM(s.net_amount) AS revenue
FROM   fact_sales s
JOIN   dim_date d ON d.date_key = s.date_key
WHERE  d.year = 2026
       AND d.is_business_day = TRUE

```

```
GROUP BY d.year_month
ORDER BY d.year_month;
```

Student

You keep using `dim_date`. What does that actually look like?

ChatGPT • AI Assistant

`dim_date` is a **pre-populated dimension with one row per calendar day** - usually a 20+ year range from a few years back to a decade ahead. Instead of computing `EXTRACT(month FROM ...)` in every query, you join to `dim_date` and get every calendar attribute you'll ever need for free.

CONCEPT CARD

What every professional `dim_date` includes:

1. **date_key** - usually integer `YYYYMMDD` (e.g., 20260706).
2. **full_date** - the actual DATE.
3. **Day parts** - *dayofmonth*, *dayofweek* (name + number), *dayofyear*.
4. **Week parts** - *weekofyear*, *iso_week*.
5. **Month parts** - *monthnumber*, *monthname*, *month_short* (e.g., 'Jul').
6. **Quarter** - *quarternumber* (1-4), *quartemame* ('Q3 2026').
7. **Year parts** - *year*, *yearmonth* (e.g., '2026-07'), *fiscalyear*.
8. **Flags** - *isweekend*, *isholiday*, *isbusinessday*, *ismonthend*, *isquarterend*.
9. **Fiscal periods** - for companies whose fiscal year isn't calendar.
10. **Prior period keys** - *samedaylastyearkey*, *samedaylastmonthkey*.

Student

What about time of day?

ChatGPT • AI Assistant

EXAMPLE CARD

For hour/minute-level analysis, use a **separate `dim\_time\_of\_day`** with one row per second (or minute, depending on grain). Then facts have both a `date_key` AND a

`time_key` . Don't mix them into `dim_date` - that would explode row counts.

🗉 ANALOGY

`dim_date` is like a **pre-printed calendar with a note for every day** - holiday, weekend, fiscal quarter, month name. Reports just look up the day; they never compute it.

⚠️ WARNING

Never use `EXTRACT(month FROM order_date)` in analytical queries when a `dim_date` exists. It defeats query optimizations, indexes, and readability.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Building a fact table with `order_date DATE` and computing month/quarter in every report. Every dashboard duplicates the logic.

Correct Way: `order_date_key INT` FK to `dim_date` . Reports join and get every calendar attribute they need.

EXAM TIP CARD

Interview classic: *"Why do warehouses use a date dimension?"* Winning answer: *"To centralize calendar attributes (fiscal periods, holidays, weekend flags) so every report shares the same definitions. It replaces messy date functions with clean joins."*

🔥 EXERCISE

Build a mini `dim_date` in your database - even just 5 years worth. Populate it with a small generator query. Any date-based analysis you write afterward becomes twice as fast to write and read.

[AI Prompt] Paste this into any AI chatbot: *"Give me DDL for a professional dim_date with all standard calendar attributes AND fiscal-year support (fiscal year starts in April). Also show a SQL INSERT that populates 3 years of rows."*

DEGENERATE, JUNK, AND ROLE-PLAYING DIMENSIONS

ChatGPT • AI Assistant

```
sql -- All three special dimension patterns in one fact table
```

```

-- Junk dimension: cross-product of all low-cardinality flags
CREATE TABLE dim_transaction_flags (
    txn_flags_key    BIGINT PRIMARY KEY,
    payment_method   VARCHAR(20),      -- card / upi / cash / wallet
    channel          VARCHAR(20),      -- online / in-store / kiosk
    order_source     VARCHAR(20),      -- web / app / marketplace
    is_gift          BOOLEAN,
    is_expedited     BOOLEAN,
    UNIQUE (payment_method, channel, order_source,
           is_gift, is_expedited)
);

-- Role-playing dim: expose dim_date under multiple names
CREATE VIEW dim_order_date    AS SELECT * FROM dim_date;
CREATE VIEW dim_shipped_date  AS SELECT * FROM dim_date;
CREATE VIEW dim_delivered_date AS SELECT * FROM dim_date;

-- Fact showing all three patterns
CREATE TABLE fact_sales (
    sales_key          BIGINT PRIMARY KEY,

    -- Role-playing: same physical dim_date used 3 times
    order_date_key     INT REFERENCES dim_date(date_key),
    shipped_date_key    INT REFERENCES dim_date(date_key),
    delivered_date_key INT REFERENCES dim_date(date_key),

    customer_key       BIGINT REFERENCES dim_customer(customer_key),
    product_key        BIGINT REFERENCES dim_product(product_key),

    -- Junk: all the small flags in one dim FK
    txn_flags_key      BIGINT REFERENCES
dim_transaction_flags(txn_flags_key),

    -- Degenerate dimension: no dim table, sits on the fact
    order_id           VARCHAR(50) NOT NULL,

    quantity           INT    NOT NULL,
    net_amount         DECIMAL(12, 2) NOT NULL
);

-- Query using role-playing (via view aliases)
SELECT od.year_month AS ordered_in,
       dd.year_month AS delivered_in,
       COUNT(*)      AS orders,
       AVG(dd.date_key - od.date_key) AS avg_days_to_deliver

```

```
FROM fact_sales s
JOIN dim_order_date od ON od.date_key = s.order_date_key
JOIN dim_delivered_date dd ON dd.date_key = s.delivered_date_key
GROUP BY od.year_month, dd.year_month;
```

Student

I've heard of **degenerate**, **junk**, and **role-playing** dimensions. What are they?

ChatGPT • AI Assistant

Three special patterns for handling dimension-like attributes that don't fit the normal "dim table with descriptive columns" mold. Each solves a specific real-world design problem.

CONCEPT CARD

The three patterns:

1. **Degenerate dimension** - a business identifier stored **directly in the fact table** with no separate dim table. Example: `order_id` - a bare identifier with no attributes to describe.
2. **Junk dimension** - a bucket table that groups **low-cardinality flags** (like `payment_type`, `channel`, `is_gift`) into one dim to avoid dozens of tiny dims.
3. **Role-playing dimension** - the **same dim table used multiple times** for different roles in a fact (e.g., `dim_date` used as both `order_date_key` and `shipped_date_key`).

Student

When should I use a junk dimension?

ChatGPT • AI Assistant

EXAMPLE CARD

Junk dim is the right call when:

1. You have many low-cardinality columns (`paymentmethod`, `channel`, `ordersource`, `isgift`, `isexpedited`...) each with 3-5 values.
2. Building a separate dim for each would clutter the model.

3. Combining them into one `dim_transaction_flags` cross-product table (payment_method × channel × ...) keeps the fact narrow.

🔗 ANALOGY

The three patterns are like different **filing solutions**:

- Degenerate = staple the receipt number to the invoice (no separate file needed).
- Junk = one "miscellaneous" folder for all small flags.
- Role-playing = one calendar hung in multiple offices, each interpreted differently.

⚠️ WARNING

Role-playing dimensions require **views or aliases** - one physical `dim_date` table, but exposed as `dim_order_date` , `dim_shipped_date` , `dim_delivered_date` when you query. Otherwise, joins get confusing.

❗ COMMON PITFALL / MISTAKE

Incorrect Way: Creating three duplicate copies of `dim_date` - `dim_order_date` , `dim_ship_date` , `dim_delivery_date` . Wastes storage; risks drift.

Correct Way: One physical `dim_date` ; use aliases or views for role-playing.

EXAM TIP CARD

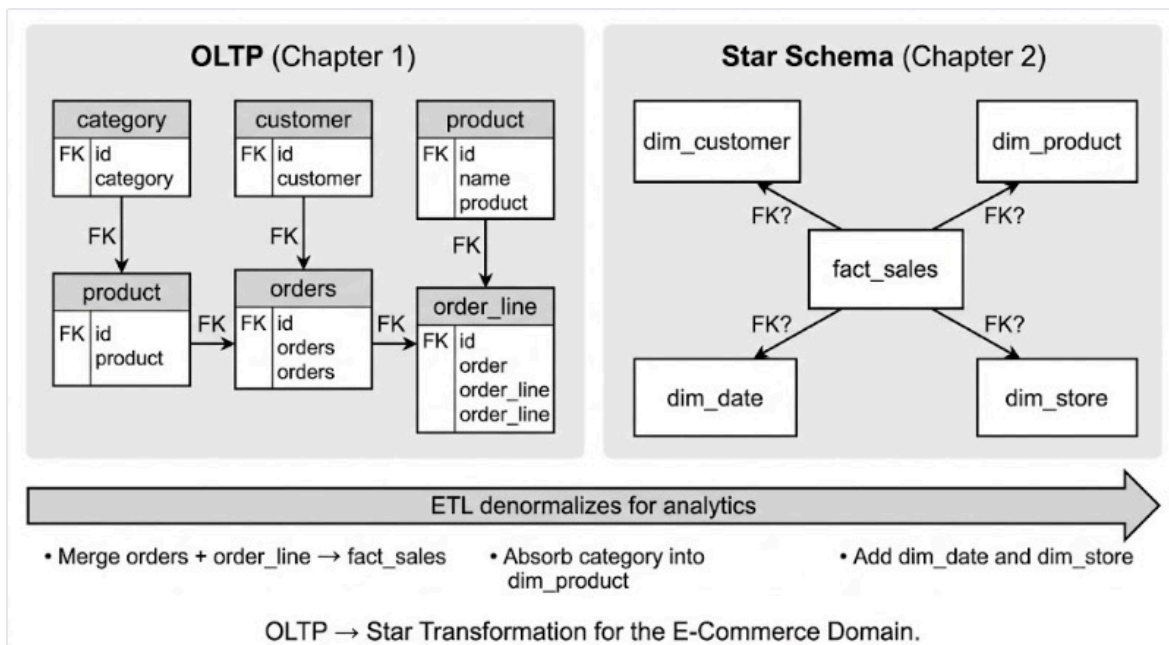
Interview favorite: *"What's a role-playing dimension?"* Winning answer: *"The same physical dim table used in multiple contexts within one fact. Common example: 'dim_date' referenced as order_date, ship_date, and delivery_date. Views or aliases give each role a friendly name."*

🔥 EXERCISE

Look at your Chapter 1 e-commerce model. Identify: (1) any degenerate dim (orderid), (2) any low-cardinality flags that could be junked, (3) any date column that could be role-played (orderdate + ship_date).

[AI Prompt] Paste this into any AI chatbot: *"For an e-commerce fact table, give me one concrete example each of a degenerate dimension, a junk dimension (with columns), and a role-playing dimension (with view aliases). Show DDL for the junk dim."*

HANDS-ON — CONVERT THE OLTP MODEL TO A STAR



OLTP → Star Transformation for the E-Commerce Domain

sql – Complete e-commerce star schema – the target model

```
-- =====
-- E-commerce star schema – the target of the OLTP-to-star ETL
-- Grain of fact_sales: one row per order line
-- =====
```

```
CREATE TABLE dim_date (
    date_key          INT          PRIMARY KEY,
    full_date         DATE          NOT NULL UNIQUE,
    day_of_week       VARCHAR(10),
    month_name        VARCHAR(10),
    quarter_name      VARCHAR(10),
    year              INT          NOT NULL,
    is_weekend        BOOLEAN      DEFAULT FALSE,
    is_holiday        BOOLEAN      DEFAULT FALSE
);
```

```
CREATE TABLE dim_customer (
    customer_key      BIGINT       PRIMARY KEY,
    customer_id       VARCHAR(50)  NOT NULL,
    full_name         VARCHAR(200),
    email             VARCHAR(200),
    city              VARCHAR(100),
    country           VARCHAR(80),
    segment           VARCHAR(50),
    first_order_date  DATE,
```

```
-- SCD Type 2 columns (Chapter 3)
effective_from DATE NOT NULL,
effective_to DATE,
is_current BOOLEAN NOT NULL DEFAULT TRUE
);

-- NOTE: category attributes are DENORMALIZED into dim_product.
-- No separate dim_category table = it's a STAR, not a snowflake.
CREATE TABLE dim_product (
    product_key BIGINT PRIMARY KEY,
    sku VARCHAR(50) NOT NULL,
    product_name VARCHAR(200) NOT NULL,
    category_code VARCHAR(20),
    category_name VARCHAR(100),
    brand VARCHAR(100),
    unit_cost DECIMAL(12, 2),
    effective_from DATE NOT NULL,
    effective_to DATE,
    is_current BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE dim_store (
    store_key BIGINT PRIMARY KEY,
    store_id VARCHAR(20) NOT NULL,
    store_city VARCHAR(100),
    region VARCHAR(50),
    store_format VARCHAR(20) -- flagship / express / online
);

CREATE TABLE fact_sales (
    sales_key BIGINT PRIMARY KEY,

    -- FKs to dimensions
    order_date_key INT NOT NULL
        REFERENCES dim_date(date_key),
    customer_key BIGINT NOT NULL
        REFERENCES dim_customer(customer_key),
    product_key BIGINT NOT NULL
        REFERENCES dim_product(product_key),
    store_key BIGINT NOT NULL
        REFERENCES dim_store(store_key),

    -- Degenerate dimensions (business IDs kept on the fact)
    order_id VARCHAR(50) NOT NULL,
    order_line_id VARCHAR(50) NOT NULL,
```

```

-- Additive measures
quantity            INT            NOT NULL,
unit_price          DECIMAL(12, 2) NOT NULL, -- non-additive
gross_amount        DECIMAL(12, 2) NOT NULL, -- quantity*unit_price
discount_amount     DECIMAL(12, 2) NOT NULL DEFAULT 0,
net_amount          DECIMAL(12, 2) NOT NULL, -- gross - discount
tax_amount          DECIMAL(12, 2) NOT NULL DEFAULT 0,
cost_amount         DECIMAL(12, 2)          -- for margin
);

CREATE INDEX idx_fs_date      ON fact_sales(order_date_key);
CREATE INDEX idx_fs_customer ON fact_sales(customer_key);
CREATE INDEX idx_fs_product  ON fact_sales(product_key);
CREATE INDEX idx_fs_store    ON fact_sales(store_key);

-- =====
-- The star pays off: a full revenue-by-category-and-month
-- report is just 2 joins.
-- =====

SELECT p.category_name,
       d.year || '-' || LPAD(d.month_name, 3, '0') AS ym,
       SUM(s.net_amount) AS revenue,
       SUM(s.quantity)   AS units_sold
FROM   fact_sales   s
JOIN   dim_product  p ON p.product_key   = s.product_key
JOIN   dim_date      d ON d.date_key      = s.order_date_key
WHERE  d.year = 2026
GROUP BY p.category_name, ym
ORDER BY p.category_name, ym;

```

Student

Can you take my **Chapter 1 e-commerce OLTP schema** and convert it to a proper Kimball star?

ChatGPT • AI Assistant

Perfect closing exercise. We'll transform the 5-table OLTP model (category, customer, product, orders, order_line) into a star schema: **one fact table + four dimensions**. This teaches you the full transformation professional data engineers do every day.

CONCEPT CARD

The transformation recipe:

1. **Choose the business process** - Sales / Order Fulfillment.
2. **Declare the grain** - one row per order line.
3. **Identify dimensions** - *dimcustomer*, *dimproduct* (with category denormalized in), *dimdate*, *dimstore* (add a store to make it interesting).
4. **Identify facts** - quantity, unitprice, lineamount, discount_amt.
5. **Add surrogate keys** to every dim; convert natural FKs in orders/order_line to surrogate FKs.
6. **Denormalize** category attributes into *dim_product* - no separate *dim_category* table.
7. **Add a `dim\_date`** with rich calendar attributes.
8. **Retain `order\_id` and `order\_line\_id` as degenerate dimensions** on the fact.

Student

How is this different from the OLTP version?

ChatGPT • AI Assistant

EXAMPLE CARD

Key differences you'll see in the DDL below:

1. **`category` table is gone** - its attributes are now columns in *dim_product*.
2. **`orders` and `order\_line` are merged** into *fact_sales* - the fact is at line-item grain.
3. **Every FK is a surrogate `\_key`** - no natural keys used for joins.
4. **`dim\_date` replaces raw DATE columns** - one FK per date role.
5. **Additive measures live on the fact**; descriptive text moved to dims.

🔗 ANALOGY

Converting OLTP to a star is like **rewriting a novel as a screenplay** - the story is the same, but the format is optimized for a different audience (analysts instead of application code).

⚠️ WARNING

The conversion is often done by **ETL/ELT pipelines** (dbt, Airflow, Fivetran + dbt) - not by manually copying data. This chapter shows the *target* schema; how you fill it is a separate discipline (Chapters 3 and 5 touch on it).

 EXERCISE

Copy the DDL below into any SQL environment. Compare it side by side with your Chapter 1 OLTP schema. Count the tables, the joins in a revenue query, and the columns per table. The star wins on all three for analytics.

[AI Prompt] Paste this into any AI chatbot: *"Take a 5-table OLTP e-commerce schema (category, customer, product, orders, order_line) and convert it to a Kimball star schema with dim_customer, dim_product, dim_date, dim_store, and fact_sales at line-item grain. Explain each choice."*

CHAPTER REVIEW & SUMMARY

Student

Can you explain what this chapter is about?

ChatGPT • AI Assistant

CHAPTER SUMMARY

This chapter taught you to think like a dimensional modeler. You now have the vocabulary, methodology, and hands-on experience to design a star schema for any business process.

- **WHAT IS THE KIMBALL METHODOLOGY:** A bottom-up, business-process-driven approach to warehouse design built on the four-step process - business process → grain → dimensions → facts.
- **FACT TABLES:** Narrow, tall tables containing FKs to dimensions plus additive/semi-additive/non-additive measures - the numeric heartbeat of the warehouse.
- **DIMENSION TABLES:** Wide, denormalized tables holding descriptive context (who/what/where/when) with surrogate PKs, natural keys, hierarchy attributes, and SCD columns.
- **GRAIN:** A one-sentence declaration of what a fact-table row represents - the pivotal design decision; prefer the finest grain you can afford; never mix grains.
- **TYPES OF FACT TABLES:** Transaction (one row per event, insert-only), periodic snapshot (one row per entity per period), accumulating snapshot (one row per process instance, updated as it progresses).

- **CONFORMED DIMENSIONS:** Dimension tables used identically across multiple facts - the foundation of the Kimball bus and the key to cross-process analysis.
- **DATE AND TIME DIMENSIONS:** Pre-populated dim_date with one row per day and every calendar attribute - replaces messy date functions with clean joins.
- **DEGENERATE, JUNK, ROLE-PLAYING:** Three special dimension patterns for business IDs on the fact, bundled low-cardinality flags, and reusing the same dim in multiple roles.
- **HANDS-ON CONVERT OLTP TO STAR:** Merged Chapter 1's 5-table OLTP model into a star with one fact *sales (line-item grain)* + four dimensions, with category denormalized into dimproduct.

Student

Can you break down the next concept?

ChatGPT • AI Assistant

You can now design analytical models the way senior data engineers do. In the next chapter we solve the biggest real-world problem in dimensional modeling: **Slowly Changing Dimensions (SCD)** - how to handle history when a customer moves, a product is recategorized, or an employee changes roles.

That's where the effective *from* / *effective to* / *is_current* columns we've been previewing finally earn their keep.